

Script-Controlled ACL - Restrict Record Access Based on Field Value

ServiceNow Micro-Project

PHASE 7 | PROJECT DOCUMENTATION PHASE

7. Project Documentation Phase

7.1 Project Summary

This project implements a **script-controlled Access Control List (ACL)** in ServiceNow to restrict users from viewing, creating, editing, or deleting records unless they meet specific role and field-value conditions. Users belonging to the EEE branch (holding custom roles bb1-bb4) can perform CRUD operations only on records where Branch = EEE, while administrators retain unrestricted access to all records in the u_institution_details table.

7.2 Technical Documentation — Table Schema

Field Label	Column Name	Type
Student Roll Number	u_student_roll_number	Auto Number
Student Name	u_student_name	Reference (User)
Faculty Name	u_faculty_name	Reference (User)
Branch	u_branch	Choice (ECE / EEE / CSE)
Email	u_email	String
Phone Number	u_phone_number	String
Description	u_description	String (multi-line)

7.3 Roles Reference

Role	Governs	Assigned To
bb1	Read (with Branch = EEE data condition)	EEE User
bb2	Create	EEE User
bb3	Write	EEE User
bb4	Delete	EEE User
admin	All operations, all branches (override)	System Administrator

7.4 ACL Configuration Reference

ACL	Type	Operation	Requires Role	Data Condition	Advanced Script
Read ACL	record	read	bb1	Branch is EEE	Yes — admin OR bb1 → allow; else deny
Create ACL	record	create	bb2	None	Not required
Write ACL	record	write	bb3	None	Not required
Delete ACL	record	delete	bb4	None	Not required

7.5 Script Documentation (Read ACL)

```
(function () {
  // Allow admin users full access
  if (gs.hasRole('admin')) {
    return true;
  }
  // Allow only EEE branch users to see EEE records
  if (gs.hasRole('bb1')) {
    return true;
  }
  // Deny access for all others
  return false;
})();
```

Line(s)	Purpose
IIFE wrapper (function(){...}) ();	Self-executing function returns a boolean directly to the ACL engine, which reads the return value as Allow (true) or Deny (false).
gs.hasRole('admin')	Checks the requesting user's roles via GlideSystem; guarantees the administrator override defined in the design phase.
gs.hasRole('bb1')	Checks for the branch-specific custom role; combined with the Data Condition (Branch is EEE) this restricts visibility to EEE rows only for bb1 holders.
return false;	Explicit default-deny — any user not matching the above conditions is refused access, preventing accidental data exposure.

7.6 How Access Is Evaluated (Recap)

1. ServiceNow first checks the ACL's **Requires role** related list — the user must hold at least one listed role to be considered at all.
2. For the Read ACL, the platform then checks the **Data Condition** (Branch is EEE) against the specific record being requested.
3. Finally, the **Advanced (script) condition** runs, returning true/false to make the final Allow/Deny decision.
4. If any layer fails, the operation is denied and the record (or capability) is hidden from the user.

7.7 Known Limitations & Future Enhancements

- Branch-to-role mapping is currently manual (bb1 tied to EEE only); a future iteration could generalize the script to read the user's own branch from their profile and compare it dynamically to the record's Branch field, supporting ECE and CSE users without new hard-coded roles.
- The four operations use four separate roles (bb1–bb4); a future version could consolidate this into fewer roles with operation-specific ACL scripts if the organization prefers a simpler role model.
- No UI Policy currently prevents a user from attempting an action they lack rights to (e.g., the New button is simply absent); explicit error messaging could be added via a Business Rule for a friendlier UX.

7.8 Outcome

By completing this project, the implementation demonstrates how script-controlled READ, WRITE, CREATE, and DELETE ACLs enforce record-level security, how user roles and record field values are evaluated together to make access decisions, and how ACLs protect sensitive data from unauthorized viewing, modification, creation, and deletion across forms, lists, and Service Portal interfaces.